

INDENG 231 Course Project 2

LLM-Powered Personal Knowledge Base on
AI Social Platforms for Shared Human Experiences

1. What Was Built

This project implements a Karpathy-style personal knowledge base using the Google Gemini API (free tier). The system has three layers: (1) a corpus of raw research notes on the project topic, written informally with deliberate gaps and contradictions; (2) an LLM-powered compiler that reads all raw notes simultaneously and synthesizes structured, interlinked wiki articles; and (3) a retrieval-augmented Q&A; interface that uses local sentence-transformer embeddings to find relevant articles and Gemini to generate cited answers.

1.1 Topic

The knowledge base covers the design of an AI-powered social platform for shared human experiences. The platform connects users who have navigated similar life challenges (illness, grief, career disruption, immigration) on a mentor-mentee model, mediated by a voice-enabled AI conversational interface. The knowledge base synthesizes five research areas: shared experience platform design, mentor-mentee matching algorithms, affective computing, voice dialogue systems (STT/TTS), and empathetic AI design.

1.2 System Architecture

The pipeline follows a raw → wiki → Q&A; flow:

Layer	Files	Description
Raw notes	raw/ (10 .md files)	Informal research notes; 33,558 chars; intentionally messy with gaps and contradictions between notes
Compiler	compile_wiki.py	Gemini 2.5 Flash reads all 10 notes in one context window; synthesizes wiki articles that resolve contradictions
Wiki	wiki/ (10 .md files)	LLM-generated, interlinked articles; one per major concept; committed to repo so Q&A works
Index	wiki_index.json	Link graph: outgoing/incoming WikiLink counts and word counts per article; rebuilt by build_index.py
Q&A interface	qa.py	sentence-transformers (all-MiniLM-L6-v2, local) embeds wiki articles and queries for cosine similarity

1.3 Raw Notes (10 Documents)

- 01: Shared experience platforms — design principles, privacy, matching quality metrics
- 02: Mentor-mentee matching — embedding-based retrieval, two-stage ranking, cold-start
- 03: Affective computing — emotion recognition, trajectory modeling, distress detection
- 04: Speech-to-text pipelines — Whisper, streaming STT, disfluency handling, privacy
- 05: Text-to-speech synthesis — neural TTS, SSML prosody control, expressive synthesis
- 06: Dialogue systems — end-to-end neural dialogue, state tracking, conversation policy
- 07: Empathetic AI design — validation-before-advice, failure modes, empathy measurement
- 08: Experience storage and retrieval — pgvector, embedding schema, privacy-preserving retrieval
- 09: Platform integration architecture — microservices, latency budget, feedback loop
- 10: LLM knowledge base design — this system's own architecture and design rationale

1.4 Compiled Wiki — Real LLM Output (10 Articles)

The compiler (Gemini 2.5 Flash) was run against the raw notes and generated all 10 wiki articles. Each article includes Key Concepts, How It Works, Design Implications, Open Questions, and See Also sections with [[WikiLinks]]. The prompt explicitly instructs the model to resolve contradictions between notes and surface implications that only emerge from reading multiple sources together — the core value of the compilation step.

Article	Words	Out Links	In Links
AffectiveComputing	719	16	6
DialogueSystem	703	14	6
EmpathyDesign	946	10	5
ExperienceMatching	373	1	4
ExperienceStorageRetrieval	409	6	7
KnowledgeBaseDesign	531	10	0
MentorMenteeSystem	239	0	5
PlatformArchitecture	669	12	3
SafetyAndEscalation	678	6	2
VoiceInterface	818	15	5

Total: 6,085 words across 10 articles, 90 outgoing WikiLinks. ExperienceStorageRetrieval has the most incoming links (7), reflecting its role as the data layer referenced by nearly every other subsystem.

2. Evaluation: What Worked and What to Improve

2.1 What Worked Well

Real LLM synthesis, not reformatting. Because the raw notes were intentionally messy and fragmented, the compiled wiki articles are genuinely better than the raw sources — not just reformatted versions. The model added Design Implications and Open Questions sections that synthesize across multiple notes, resolving tensions (e.g., when to use keyword vs. embedding retrieval) that no single note addressed.

Embedding retrieval handles semantic queries. Using sentence-transformers (all-MiniLM-L6-v2, local, no API cost) for retrieval correctly surfaces semantically relevant articles that keyword matching would miss. The query 'how does the system detect sadness or distress?' retrieves AffectiveComputing at 0.615 cosine similarity — not by keyword overlap, but by meaning.

WikiLink graph reveals knowledge structure. The compiler generated 90 WikiLinks across 10 articles without explicit instruction on which topics to connect. ExperienceStorageRetrieval emerged as the most-referenced article (7 incoming links), correctly identifying it as the shared data layer for matching, affective computing, and dialogue systems.

Fully free pipeline. The entire system runs at zero cost: Gemini 2.5 Flash free tier for compilation and Q&A;, sentence-transformers for local embedding, no paid APIs required. This makes the pipeline reproducible for any course project or personal knowledge base.

Topic-agnostic modularity. Replacing the raw/ directory with notes on any domain and rerunning compile_wiki.py produces a working knowledge base with no code changes.

2.2 Limitations and What to Improve

Uneven article quality. Some articles (EmpathyDesign: 946 words, VoiceInterface: 818) are thorough while others (MentorMenteeSystem: 239, ExperienceMatching: 373) are thin. This reflects uneven depth in the raw notes. The fix is richer raw notes for underrepresented topics, and a minimum word-count check in the compiler that retries short articles.

Single-hop retrieval. The Q&A; interface retrieves the top-4 articles by embedding similarity but does not follow WikiLinks to transitively relevant articles. Questions spanning multiple concepts would benefit from graph-walk retrieval that follows links from the initial result set.

No incremental compilation. Adding a new raw note requires recompiling all articles. A smarter compiler would classify which wiki articles are affected by a new source and only recompile those, reducing cost and latency for knowledge base updates.

No evaluation benchmark. Compilation quality was assessed qualitatively. A rigorous evaluation would include a held-out question set with ground-truth answers derivable from the raw notes, measuring answer accuracy, citation precision, and hallucination rate against the wiki.

WikiLinks not navigable in Q&A; output. The Q&A; interface renders `[[WikiLink]]` as bold text. A simple web frontend (Flask + markdown rendering) would make links clickable, enabling true wiki-style navigation — the most natural way to explore the knowledge graph.